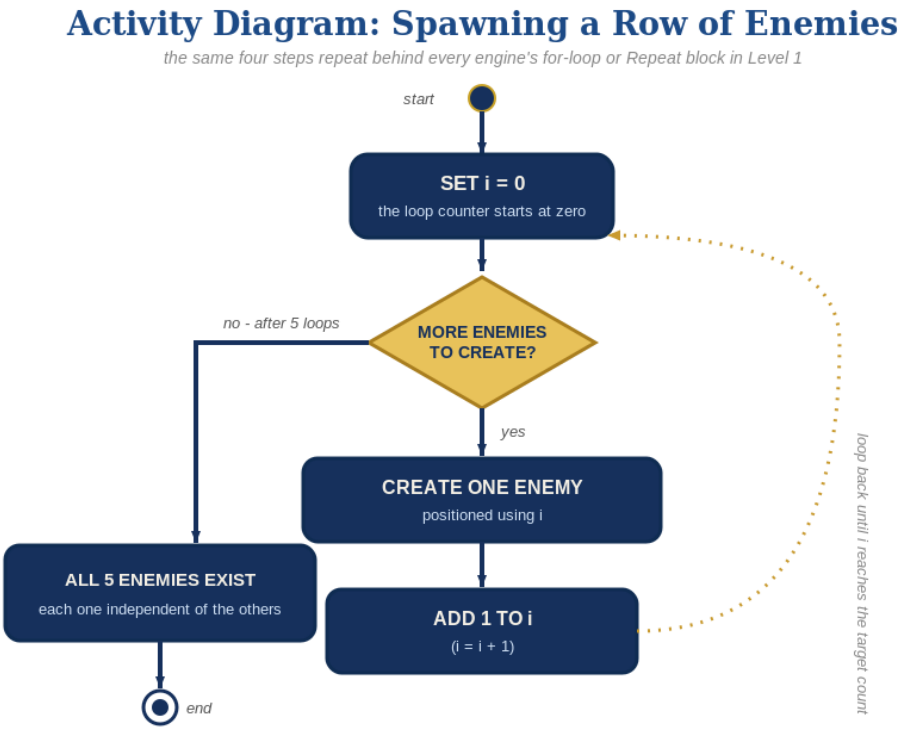
Lesson 5 Teacher's Key

TIC-80

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution



Every engine's Level 1 solution runs this same shape - a counter, a repeat check, one creation step, and an increment - even though the exact blocks or code syntax differ. Tracing a spawn bug means finding which of these four steps broke.

The spawn loop, in general - the same counter, repeat check, creation step, and increment behind Level 1 in every engine, whatever the exact blocks or syntax look like.

An enemies table populated by a for loop at startup, plus a nested loop inside the existing bullets loop:

```
enemies={}
for enemy_count=0,4 do
 table.insert(enemies,{ex=20+enemy_count*20,ey=10,alive=true})
end
bullets={}
```

```
function TIC()
 cls(0)
 if btn(3) then x=x+1 end
 if btnp(4) then
  table.insert(bullets,{bx=x,by=y})
 end
 for i=#bullets,1,-1 do
  local b=bullets[i]
  b.by=b.by-2
  local hit=false
  for j=1,#enemies do
   local e=enemies[j]
   if e.alive and b.bx<e.ex+8 and b.bx+8>e.ex and b.by<e.ey+8 and
b.by+8>e.ey then
    e.alive=false
    hit=true
   end
  end
  if hit then
   table.remove(bullets,i)
  elseif b.by<0 then
   table.remove(bullets,i)
  else
   spr(2,b.bx,b.by,0)
  end
 end
 for j=1,#enemies do
  local e=enemies[j]
  if e.alive then spr(3,e.ex,e.ey,0) end
 end
 spr(1,x,y,0)
end
```

Level 2 - Complete Worked Solution

The inner enemies loop checks every enemy for every bullet, every frame, so this naturally scales to any number of enemies with no extra code - a hit sets e.alive=false for exactly the one enemy whose bounds actually overlapped that bullet.

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: already works

sfx(2) can stay exactly where it was added in Lesson 4's hit branch - add it inside the hit=true assignment if not already there:

```
   if e.alive and b.bx<e.ex+8 and b.bx+8>e.ex and b.by<e.ey+8 and
b.by+8>e.ey then
    e.alive=false
    hit=true
    sfx(2)
   end
```

Visual: row variety

Use each enemy's index j to flip alternating enemies for a bit of visual variety:

```
for j=1,#enemies do
 local e=enemies[j]
 if e.alive then spr(3,e.ex,e.ey,0,1,j%2) end
end
```

Structure: enemy state lives in the table

Every place that needs an enemy's position or status reads it from the enemies table - resist ever declaring a loose ex/ey/alive variable outside it again.

Complete Level 3 solution (all 3 goals combined)

Everything above combined into the one TIC() function:

```
enemies={}
for enemy_count=0,4 do
 table.insert(enemies,{ex=20+enemy_count*20,ey=10,alive=true})
end
bullets={}

function TIC()
 cls(0)
 if btn(3) then x=x+1 end
 if btnp(4) then
  table.insert(bullets,{bx=x,by=y})
 end
 for i=#bullets,1,-1 do
  local b=bullets[i]
  b.by=b.by-2
  local hit=false
  for j=1,#enemies do
   local e=enemies[j]
   if e.alive and b.bx<e.ex+8 and b.bx+8>e.ex and b.by<e.ey+8 and
b.by+8>e.ey then
    e.alive=false
    hit=true
    sfx(2)
   end
  end
  if hit then
   table.remove(bullets,i)
  elseif b.by<0 then
   table.remove(bullets,i)
  else
   spr(2,b.bx,b.by,0)
  end
 end
 for j=1,#enemies do
  local e=enemies[j]
  if e.alive then spr(3,e.ex,e.ey,0,1,j%2) end
 end
 spr(1,x,y,0)
end
```